

Universidad San Sebastián

Taller 1:

Desarrollo de un Intérprete de Comandos (Shell) en Linux

Sistemas Operativos — Primer Semestre 2026

Profesor: Juan Felipe González Saavedra

Integrantes:

Ignacio Barra
Carlos Bastidas
Javier Morales
Matías Salazar

Mayo 2026

Resumen

Este informe presenta el desarrollo de myshell, una shell básica para sistemas Unix/Linux implementada en lenguaje C como parte del Taller 1 de Sistemas Operativos. Para su construcción se utilizaron las llamadas a sistema `fork()`, `execvp()` y `waitpid()` para la ejecución de procesos, junto a `write()`, `fgets()` y `exit()/_exit()` para el resto de las funcionalidades. El programa cumple con los seis requerimientos del taller: muestra un prompt de espera, lee y procesa comandos con cualquier cantidad de argumentos, permite salir con el comando `exit`, informa cuando un comando no existe sin cerrar la shell, vuelve a mostrar el prompt si el usuario presiona Enter sin escribir nada, y ejecuta los comandos en foreground esperando su término antes de continuar. Todas las funcionalidades fueron verificadas y operan de forma correcta.

Introducción

Una shell es el programa que conecta al usuario con el sistema operativo: recibe los comandos escritos en el terminal y se encarga de ejecutarlos. Herramientas como Bash o Zsh son de uso cotidiano en Linux, y entender cómo funcionan por dentro es fundamental para comprender cómo el sistema operativo gestiona los procesos.

En el Taller 1 de Sistemas Operativos (Primer Semestre 2026) se pidió diseñar e implementar una shell básica en C, identificando qué llamadas a sistema son necesarias para cada funcionalidad. Este documento explica las decisiones tomadas durante el desarrollo, describe el código implementado y muestra evidencias de que el programa funciona correctamente. El código fuente y las instrucciones de compilación están disponibles en el siguiente repositorio: <https://github.com/SubaruDev0/sistemasoperativos.git>

El objetivo principal del taller es que los estudiantes se familiaricen con la creación y manejo de procesos en Unix/Linux usando `fork()`, `exec()` y `wait()`, además de conocer otras herramientas del sistema como señales y pipes.

Marco Teórico

Llamadas a Sistema en Unix/Linux

Una llamada a sistema es la forma en que un programa le pide al sistema operativo que haga algo por él, como crear un proceso o leer desde el teclado. A continuación se describen las más relevantes para el desarrollo de una shell en Unix/Linux.

Gestión de procesos

`fork()`: crea un nuevo proceso hijo como copia del proceso padre. Retorna 0 al hijo y el PID del hijo al padre. Es la base para ejecutar cualquier comando en la shell.

`execvp()`: reemplaza el proceso actual con el programa que se quiere ejecutar, buscándolo en los directorios del PATH. Si funciona, no retorna; si retorna, es porque ocurrió un error.

`waitpid()`: hace que el proceso padre espere a que un hijo específico termine. Gracias a esto la shell puede funcionar en foreground, mostrando el prompt solo cuando el comando ya finalizó.

`_exit()`: cierra el proceso sin ejecutar rutinas de limpieza. Se usa en el proceso hijo cuando `execvp()` falla, para evitar que el hijo siga ejecutando código que corresponde al padre.

Entrada/salida

`write()`: escribe datos directamente en un descriptor de archivo. Se usa para mostrar el prompt en pantalla sin pasar por buffers intermedios, de modo que aparezca al instante.

`fgets()`: lee una línea completa desde stdin usando internamente `read()`. Es la función con la que la shell captura lo que escribe el usuario.

Terminación del proceso

`exit()`: cierra el proceso de forma ordenada, ejecutando las rutinas de limpieza y vaciando los buffers. Se usa en el proceso padre para terminar la shell correctamente cuando el usuario escribe `exit`.

Modelo Fork-Exec-Wait

El modelo fork-exec-wait es la forma estándar de ejecutar programas en Unix/Linux. El padre usa `fork()` para crear un hijo; el hijo usa `exec()` para convertirse en el programa que se quiere correr; y el padre usa `waitpid()` para esperar a que el hijo termine. Este esquema permite que la shell siga funcionando mientras los comandos se ejecutan en procesos separados (Silberschatz et al., 2018).

Desarrollo e Implementación

Arquitectura General

myshell funciona como un ciclo repetitivo que sigue estos pasos: (1) mostrar el prompt, (2) leer lo que escribe el usuario, (3) separar la línea en el comando y sus argumentos, (4) revisar si se trata de `exit` o una línea vacía, y (5) ejecutar el comando con `fork/exec/wait`. El ciclo termina cuando el usuario escribe `exit` o presiona `Ctrl+D`.

Descripción por Ítem

Ítem 1 — Prompt de espera

Llamada a sistema utilizada: `write()`. El texto `myshell>` se imprime en pantalla usando `write()` directamente sobre `stdout`. Se prefirió esta llamada sobre `printf()` o `puts()` porque no usa buffers intermedios, lo que garantiza que el prompt aparezca de inmediato.

```
write(STDOUT_FILENO, PROMPT, strlen(PROMPT)); /* PROMPT =  
"myshell> " */
```

Ítem 2 — Lectura y parseo de comandos

Llamada a sistema utilizada: `read()` (a través de `fgets()`). `fgets()` captura la línea completa escrita por el usuario, hasta un máximo de 1024 caracteres. Luego, `parse_input()` la divide en el comando y sus argumentos usando `strtok()`, separando por espacios, tabuladores y saltos de línea. Esto permite manejar cualquier cantidad de argumentos (hasta 127).

```
fgets(input, sizeof(input), stdin);  
int argc = parse_input(input, args); /* usa strtok()  
internamente */
```

Ítem 3 — Comando exit

Llamada a sistema utilizada: `exit()`. Si el primer token del comando es `exit`, la shell imprime un mensaje de despedida y llama a `exit(EXIT_SUCCESS)`, que cierra el proceso de forma ordenada liberando los recursos del programa.

```
if (strcmp(args[0], "exit") == 0) {  
    write(STDOUT_FILENO, "Saliendo de myshell...\n", 23);  
    exit(EXIT_SUCCESS);  
}
```

Ítem 4 — Manejo de comandos inexistentes

Llamada a sistema utilizada: `execvp()`. Cuando `execvp()` retorna, significa que el comando no existe o no se pudo ejecutar. En ese caso el proceso hijo muestra el mensaje de error con `strerror(errno)` y termina con `_exit()`. El proceso padre retoma el control gracias a `waitpid()`, así la shell sigue funcionando con normalidad.

```
execvp(args[0], args);  
fprintf(stderr, "myshell: %s: %s\n", args[0], strerror(errno));  
_exit(EXIT_FAILURE);
```

Ítem 5 — Enter sin comando

No requiere llamada a sistema adicional. Se verifica que el primer carácter de la cadena leída sea `'\n'` (línea vacía). De ser así, se ejecuta `continue`, lo que regresa al inicio del bucle y reimprime el prompt sin procesar ningún comando.

```
if (input[0] == '\n') {  
    continue; /* volver al inicio: nuevo prompt */  
}
```

Ítem 6 — Ejecución concurrente en foreground

Llamadas a sistema utilizadas: `fork()`, `execvp()` y `waitpid()`. El padre crea un hijo con `fork()`; el hijo ejecuta el comando con `execvp()`; el padre espera con `waitpid()` hasta que el hijo termina. Así se logra el comportamiento `foreground`: la shell no vuelve a mostrar el prompt hasta que el comando haya finalizado por completo.

```
pid_t pid = fork();  
if (pid == 0) {
```

```
    execvp(args[0], args);    /* proceso hijo */
    _exit(EXIT_FAILURE);
} else {
    waitpid(pid, &status, 0);    /* padre espera */
}
```

Evidencias de Ejecución

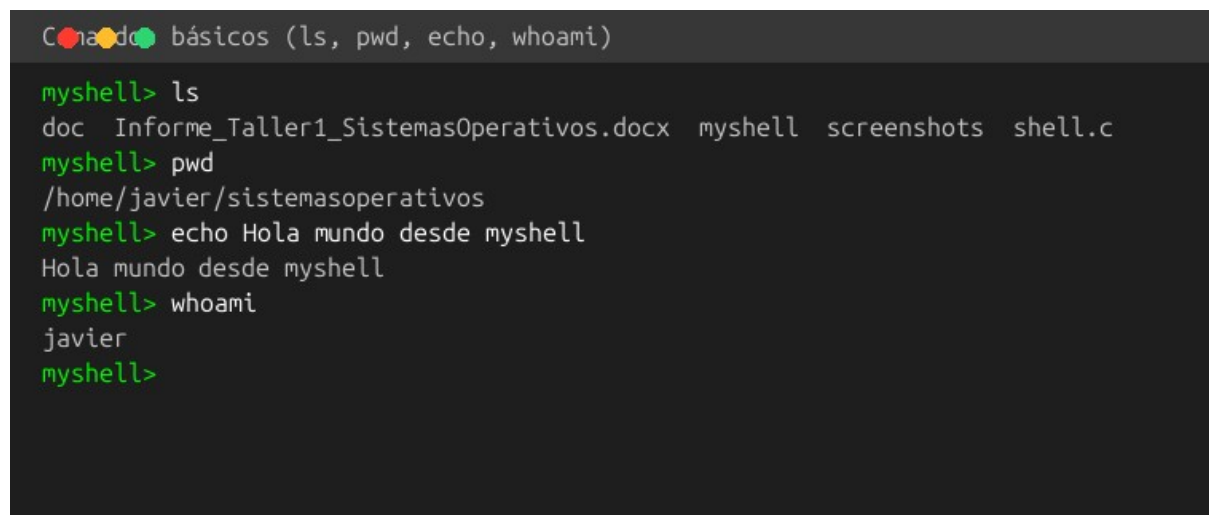
Compilación

El código fuente se compiló sin errores ni advertencias con el siguiente comando:

```
gcc -Wall -Wextra -o myshell shell.c
```

Ítems 1, 2 y 5 — Prompt, parseo y Enter vacío

Al iniciar myshell, el prompt myshell> aparece de inmediato. Presionando Enter sin ingresar ningún texto, el prompt se vuelve a mostrar sin error. Al ingresar un comando con múltiples argumentos como echo uno dos tres, la shell los procesa y ejecuta correctamente.



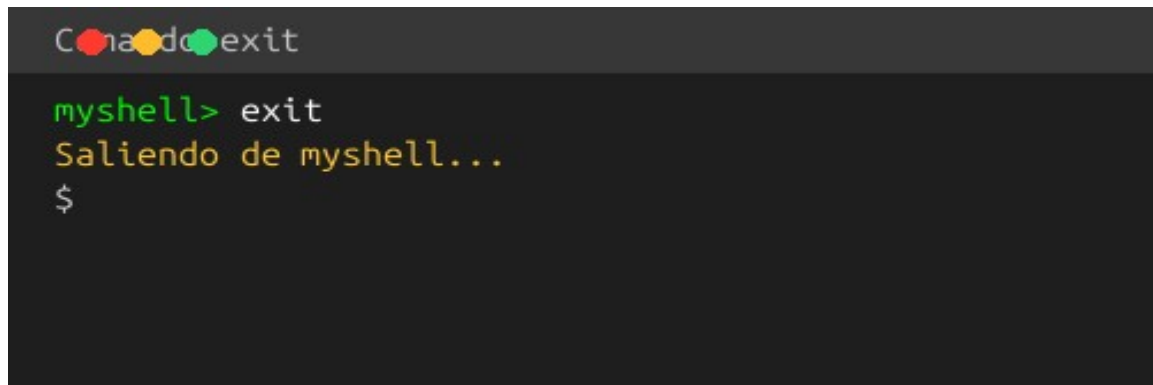
```
Comandos básicos (ls, pwd, echo, whoami)
myshell> ls
doc Informe_Taller1_SistemasOperativos.docx myshell screenshots shell.c
myshell> pwd
/home/javier/sistemasoperativos
myshell> echo Hola mundo desde myshell
Hola mundo desde myshell
myshell> whoami
javier
myshell>
```

[Figura 1: Captura de pantalla — prompt, parseo y línea vacía]

Nota. Demostración de los ítems 1, 2 y 5 en ejecución.

Ítem 3 — Comando exit

Al ingresar exit, la shell muestra el mensaje "Saliendo de myshell..." y termina el proceso, devolviendo el control al terminal del sistema operativo anfitrión.



```
Conando exit

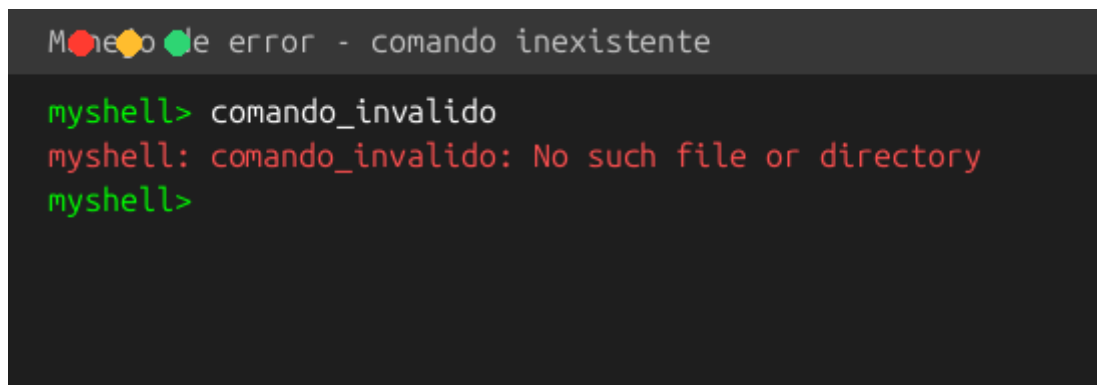
myshell> exit
Saliendo de myshell...
$
```

[Figura 2: Captura de pantalla — comando exit]

Nota. Terminación limpia de myshell.

Ítem 4 — Comando inexistente

Al ingresar un comando inexistente (por ejemplo, comandofalso), la shell muestra el mensaje de error myshell: comandofalso: No such file or directory y continúa ejecutándose, presentando nuevamente el prompt para recibir el siguiente comando.



```
Manejo de error - comando inexistente

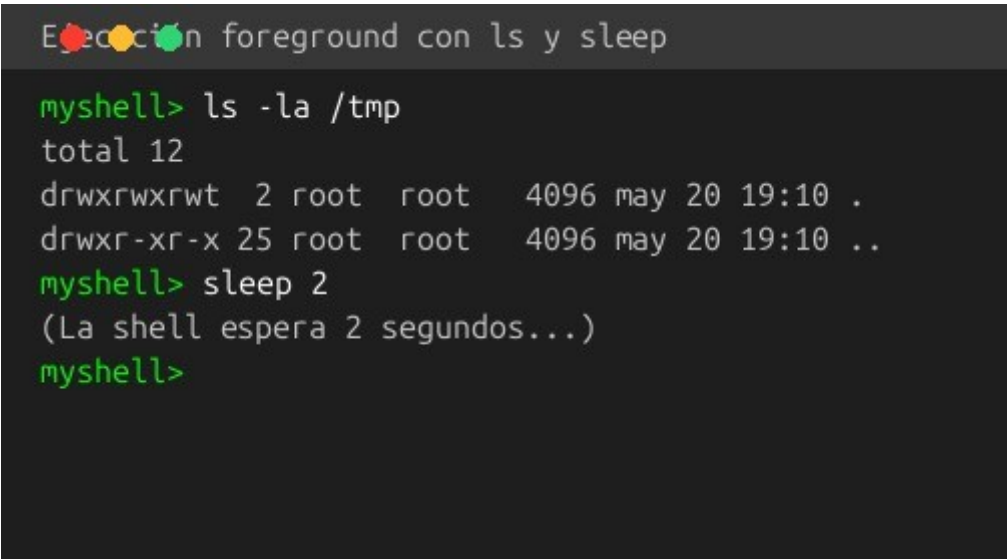
myshell> comando_invalido
myshell: comando_invalido: No such file or directory
myshell>
```

[Figura 3: Captura de pantalla — comando inexistente]

Nota. Manejo de error sin detener la shell.

Ítem 6 — Ejecución en foreground

Al ejecutar comandos como `ls -la /tmp` o `sleep 2`, la shell aguarda a que el proceso hijo termine antes de mostrar el prompt nuevamente. Esto confirma el comportamiento foreground implementado con `waitpid()`.



```
Ejecución foreground con ls y sleep

myshell> ls -la /tmp
total 12
drwxrwxrwt  2 root  root   4096 may 20 19:10 .
drwxr-xr-x 25 root  root   4096 may 20 19:10 ..
myshell> sleep 2
(La shell espera 2 segundos...)
myshell>
```

Nota. Comportamiento foreground con fork()/execvp()/waitpid().

Nota: Las capturas de pantalla completas de cada prueba están disponibles en el repositorio del proyecto, en la carpeta /screenshots/.

Conclusiones

Desarrollar myshell fue una buena forma de entender en la práctica cómo funciona el modelo fork-exec-wait en Unix/Linux. Los seis requerimientos del taller fueron implementados correctamente, y en cada caso se identificó y justificó la llamada a sistema correspondiente.

El ítem más complejo fue el 6, porque implica coordinar correctamente tres llamadas a sistema distintas para lograr el comportamiento foreground. También fue importante manejar bien los errores de execvp() y usar _exit() en el hijo cuando algo falla, para que la shell no quede en un estado inconsistente.

En términos generales, usar llamadas a sistema directas como write(), fork(), execvp(), waitpid() y exit() resultó en una shell simple pero funcional, sin depender de bibliotecas externas. Lo aprendido en este taller sirve como punto de partida para incorporar funciones más avanzadas como redirecciones, pipes, ejecución en background y manejo de señales.

Referencias

Kerrisk, M. (2010). The Linux programming interface: A Linux and UNIX system programming handbook. No Starch Press.

Linux man-pages project. (2023). fork(2), execvp(3), waitpid(2), write(2), exit(3). <https://man7.org/linux/man-pages/>

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

The Open Group. (2018). The Open Group Base Specifications Issue 7 (IEEE Std 1003.1-2017). <https://pubs.opengroup.org/onlinepubs/9699919799/>